

COURSE DESCRIPTION

Course number	0418-142	Course Title	Computer Programming II
Credit hours	4 (3:3:4)	Course Coordinator	Dr. Hamdi Yahyaoui
Compulsory	Yes	Prerequisite	0418-141

Textbook

<i>Title:</i>	<i>C++ How To Program (Late Objects Version)</i>
<i>Author:</i>	<i>Paul Deitel and Harvey Deitel</i>
<i>Publisher:</i>	<i>Pearson, Latest Edition</i>

Major Topics Covered in the Course

	Topic	Hours
1.	Basic Object Oriented Concepts (Classes, Objects, and Data Encapsulation).	3
2.	Classes: data members, member functions and overloading, constructors, copy constructors, destructors, garbage collection, abstract and nested classes.	6
3.	Composition, Objects as function parameters and return values.	4
4.	Inheritance, Multiple Inheritance, and Virtual Functions/Classes.	6
5	Friend Classes and Functions, Polymorphism.	3
6.	Exception Handling.	3
7.	Text and Binary Files, Sequential and Random Access.	6
8.	Dynamic data structures: Linked Lists, Stacks and Queues.	4
9.	Templates and STL Library (API).	2
10.	Data Structures using STL: vectors, stacks, and queues.	2
	Midterm I and Midterm II	3
	Total	42

Laboratory work:

	Topic	Week
1.	Programming with classes, data members, constructors, member functions, destructors, information hiding	1
2.	Overloading and nested classes	2
3.	Objects as function parameters and return values	3
4.	Composition	4
5.	Graphical User Interfaces (GUI) Design and Event-Driven Programming	5
6.	Inheritance, virtual functions, and classes	6
7.	Friend classes and functions, Polymorphism	7
8.	GUI Programming and Graphics	8
9.	Exception handling	9
10.	Text and Binary Files, Sequential and Random Access.	10 & 11
11.	Dynamic data structures.	12
12.	Templates and STL Library.	13
13.	Data Structures using STL.	14

Grading

20%	Midterm 1
20%	Midterm 2
15-20%	Lab
0-5%	Hmk/ quiz
40%	Final

Course Outcomes

Students should be able to:

1. Analyze, design, implement, test, and debug programs that use all constructs covered in this course.
2. Explain the importance of and the involved mechanics of fundamental concepts in object-oriented programming including encapsulation, information hiding, inheritance, polymorphism, event-based programs.
3. Utilize templates in implementing generic code and use libraries
4. Use APIs to create and display graphics.
5. Design and implement a graphical user interface.
6. Decompose a program into manageable, well-defined classes (domain-specific types). Properly define the public/private parts of each class. Define class hierarchies.

Relationship between Course Outcomes and Student Outcomes

Course Outcome	Student Outcomes					
	1	2	3	4	5	6
1.	H	H				
2.		H				
3.		H				
4.		H				
5.		H				
6.	H	H				H

Last updated on Dec 8, 2019